# Speaking script: Forkable Sandboxes

Cambridge SRG, 15 October 2026. 31 slides, **27:45** planned.

Cues: **[CLICK]** advance one build step **[ASK]** question to the room [PAUSE] two beats of silence **bold** = stress

## Cue card

| # | Starts | Land this |
|---|---|---|
| 1 | 0:00 | Forkable sandboxes let computers explore alternatives without doing work twice. |
| 2 | 0:35 | Each stop on my path left one lesson. |
| 3 | 1:30 | Self-driving compute: many runs from one state, one result. |
| 4 | 2:15 | Three steps remove it: cache, fork, sandbox. |
| 5 | 3:10 | The key must cover every input; the sandbox hides the rest. |
| 6 | 3:55 | Distributed builds rely on it: one machine reuses another's output. |
| 7 | 4:35 | A fork extends caching from files to a reached execution state. |
| 8 | 5:20 | A sandbox is a policy; a VM or a container is one way to build it. |
| 9 | 6:10 | The forkable API prepares once and runs alternatives in private children. |
| 10 | 6:55 | New: count shared evidence once; publish checked bytes once. |
| 11 | 7:45 | You will leave with all three. |
| 12 | 8:35 | Set up once; count each bug once. |
| 13 | 9:35 | One start, one fixed evaluator. |
| 14 | 10:35 | Coverage is not selection: one patch ships. |
| 15 | 11:35 | Six guides, one gene. |
| 16 | 12:35 | Two plotted branches, one family. |
| 17 | 13:35 | Compute once. Count once. Publish once. |
| 18 | 14:20 | The next step decides which state to reuse. |
| 19 | 15:25 | Price a restore against a warm cache, not a cold start. |
| 20 | 16:40 | Record effects once. Keep the publish key outside every fork. |
| 21 | 17:45 | A receipt is evidence's cache entry, written outside the child. |
| 22 | 18:50 | Each fan-in operation needs its own check. |
| 23 | 19:50 | A shared observation counts once. |
| 24 | 20:40 | Shared errors count for less. |
| 25 | 21:30 | Check outside the child, including its error bars. |
| 26 | 22:25 | Our factory's patch passed every check it had. |
| 27 | 23:20 | Accept only bytes whose promise was tested. |
| 28 | 24:10 | Save outside the child before cleanup. |
| 29 | 24:55 | Proposed: one controller, current epoch, publish once. |
| 30 | 25:40 | Five questions remain open; the preregistered tests have not started. |
| 31 | 26:45 | Compute once. Count once. Publish once. |

# 1 Motivation 0:00–8:35

## 01 Forkable sandboxes let computers explore alternatives without doing work twice.

0:35, starts 0:00

My path runs from self-organization in cells, to self-driving cars, to self-driving compute.
Say we have paid to reach a useful state.
Can we try several next steps from there without paying for it again?
My answer has three rules.
**Compute once**.
**Count once**.
**Publish once**.
The corner tags mark what is published, measured, built or only designed.
First, where these ideas come from.

## 02 Each stop on my path left one lesson.

0:55, starts 0:35

Here is my path.
Each stop left one lesson.
My early systems jobs, including Twistlock, were in software and container security.
The lesson: a sandbox bounds what a program may do.
From twenty sixteen I simulated cell mechanics and analyzed genomes.
In both, reaching a state is the expensive part.
So you explore from there.
Our twenty nineteen study tested drug combinations on cancer cells.
Many trials, few confirmations.
At NRGene, gene-editing results helped pick the next experiment.
At Mobileye, repeated observations of one road had to count once.
At Incredibuild, the rule is simple.
Never build the same thing twice.
These lessons come back later, starting with Mobileye.

## 03 A car fleet fans out, then fans in to one map.

0:45, starts 1:30

Mobileye's Roadbook is a fleet example.
Assume three hundred million equipped cars sending anonymized road data.
The cloud matches drives of the same road and combines them.
It sends the updated map back to the cars.
My own work there was junction perception.
The map counts each road once, however many cars drove it.
**Count once**.
Self-driving compute gives execution the same shape.
Many runs from one reached state, one combined result.
But a fleet of runs keeps doing the same work.
That is the problem.

## 04 In parallel computing, the costly work is the work done twice.

0:55, starts 2:15

Here is that problem in one picture.
**[ASK]** A question for you: how much of your last CI run had another machine already done? [PAUSE]
Often it is the same install and the same compile, again.
Without reuse, three trials each repeat the same setup.
**[CLICK]** With reuse, we set up once and save that state.
Every trial starts from it.
The setup blocks we skip are the waste.
That is N minus one setups.
In a parallel, distributed world, the expensive thing is doing work twice. [PAUSE]
Three steps remove it: cache, fork, sandbox.
**Step one** is the oldest: a cache.

## 05 A cache skips work a previous run already did.

0:45, starts 3:10

Here is how a cache works.
Hash a build step's inputs into a key: source, flags and toolchain.
If the key is stored, that is a hit.
Reuse the stored output.
On a miss, run the step in a sandbox.
Store its output, so the next run hits.
The sandbox hides undeclared files, so the step cannot read what the key omits.
This is the Incredibuild lesson: never build the same thing twice.
Remember that hash, because it comes back.
What does it buy?

## 06 Reusing an earlier runner's build files cut a Tokio compile from about 46 s to 13 s.

0:40, starts 3:55

In my Tokio demo, a fresh compile took about forty-six seconds.
Copying in an earlier runner's build files cut that to about thirteen.
An already warm cache cut it to about three.
These are approximate, self-reported times.
The copied files are build outputs, not a running program.
Distributed builds rely on this: one machine reuses another's output.
A cache reuses one step's output.
**Step two** reuses a whole state.

## 07 A fork extends caching from files to a reached execution state.

0:45, starts 4:35

A fork goes further.
It restores the state after many steps, like installed dependencies or a warmed process.
Think of a film.
Its scenes are fixed, and rewinding replays the same plot.
An experiment is different.
It saves a reached state, keyed by its hash, like a cache entry.
It tries two next actions, each in its own private sandbox.
A check keeps one outcome and discards the other.
Then we continue from there.

**Step three** makes that sharing safe.

## 08 A sandbox lets children share one state safely.

0:50, starts 5:20

In our design, three children share one saved state.
It is read-only and named by its hash.
Each child writes only to its private layer, so siblings cannot overwrite each other.
Hiding undeclared inputs keeps the cache key complete.
Effects, like a network request, cross the boundary.
No restore rewinds them, so we propose recording them.
This is the Twistlock lesson: a sandbox bounds what a program may do.
It is a policy.
One way to build it is a VM with its own kernel.
Another is a container on a shared kernel.
Here are the three steps as code.

## 09 The forkable API prepares once and runs alternatives in private children.

0:45, starts 6:10

Both columns are Python.
Same three edits, same test, same selection rule.
On the left, a loop prepares a fresh environment for every edit.
On the right, the proposed API prepares once and checkpoints that state.
Fork makes three children, each a private sandbox.
Each applies one edit and returns its result with evidence.
So prepare() runs once, not three times.
Ordinary Python can already do this, with process forks or caches.
The API makes reuse and isolation explicit.
None of these pieces is new on its own.

## 10 Caches, forks and sandboxes are old; coupling them to evidence and acceptance is new.

0:50, starts 6:55

A cache keeps run outputs, as in ccache, Bazel's remote cache or Incredibuild.
A fork keeps reached states.
SnowFlock cloned Xen machines in two thousand nine.
Xen came from this lab.
A sandbox keeps writes private, as containers, gVisor and microVMs do.
What is new is what sharing requires.
Count shared evidence once, and publish checked bytes once.
I claim no fork speedup, since our agent runs were serial.
I also do not claim our snapshots keep live memory.
So what does sharing work cost us?

## 11 Sharing work creates three rules.

0:50, starts 7:45

Sharing work creates three rules.
They organize this talk.
**Compute once**: reuse or rebuild?

Reuse a saved state only if it beats a rebuild with a warm cache.
That needs a cost rule.
**[CLICK] Count once**: what must a trial return?
Parallel trials share inputs and errors, so a shared observation counts once.
That needs an evidence contract.
**[CLICK] Publish once**: who publishes what?
One controller publishes the exact bytes that passed the check.
That needs an acceptance gate.
You will leave with all three.
Five examples first, then the rules in order.

# 2 Examples 8:35–14:20

## 12 Reuse makes fuzzing fast; only checks turn crashes into findings.

1:00, starts 8:35

First example: software security.
A fuzzer runs a program on many changed inputs, hunting for crashes.
AFL-plus-plus initializes the target once, then reuses one child process across many inputs.
Its documentation reports typical speedups of ten to twenty times.
It skips a fork for every input.
That works only if state resets correctly.
That is rule one, compute once.
But speed only produces leads, and leads need checks.
Berkeley's CyberGym is a separate study.
There, an AI agent wrote test inputs for 431 projects.
It produced fifty-six crashes.
The researchers reproduced them and removed duplicates, leaving twenty-two distinct new vulnerabilities.
That is rule two, count once.
Second example: AI research.

## 13 Autoresearch kept 23 of 126 attempts against a fixed evaluator.

1:00, starts 9:35

Karpathy's autoresearch starts every attempt from the current best commit.
The commit hash names that exact state.
The evaluator stays fixed.
The agent edits the training script and trains for five minutes.
It reads validation bits per byte, where lower is better.
Then it keeps or reverts.
One reported session made 126 attempts, one at a time, and kept twenty-three.
The score fell 2.8 percent.
Starting from the best commit is compute once.
Only the fixed evaluator promotes a change, and that is publish once.
AlphaEvolve extends this loop to a population of programs.
One scheduling heuristic it found recovered 0.7 percent of Google's compute, on average.
Third example: software engineering.

## 14 250 samples raised SWE-bench Lite coverage from 15.9% to 56%.

1:00, starts 10:35

The shared state is now a repository with an open issue.
Brown and colleagues sampled patches from DeepSeek-Coder-V2-Instruct on SWE-bench Lite.
With one sample, a correct patch appeared for 15.9 percent of tasks.
With 250 samples, for 56 percent.
That is coverage: at least one patch passes the hidden tests.
In production there are no hidden tests.
A selector must pick one patch from many wrong ones.
Only that patch ships.
Coverage is not selection.
That is rule three, publish once.
You will see our own software factory in the acceptance section.
There, the suite was green but the check was incomplete.
Fourth example: from code to cells.

## 15 Our CRISPR screen narrowed 19,050 genes to about 100 candidates.

1:00, starts 11:35

This one is my own work with coauthors.
The shared state is one pooled cell screen covering 19,050 genes.
The alternatives are over a hundred and twenty thousand guide RNAs.
Six per gene, across nine pooled arms.
Two ranking methods narrowed nineteen thousand genes to about one hundred.
Here is the counting lesson.
Six guides are six looks at one gene, not six discoveries.
So the gene counts once, like a road on the map.
That is rule two.
Then the check: focused assays in MCF-seven breast cancer cells.
Ten of thirteen targets increased sensitivity to the SRC-3 inhibitor SI-twelve.
Six of eight drug combinations improved killing.
These are laboratory results, not clinical ones.
Fifth example: physics.

## 16 With Ori Chamo, we connected two apparent branches of one orbit family.

1:00, starts 12:35

Ori Chamo and I studied the three-body problem.
The shared state is a catalog.
Li, Li and Liao tabulated 135,445 periodic three-body orbits and described one family.
A later analysis plotted the same catalog and saw two branches.
Our alternatives are continuations.
We change the masses in small steps, then correct each orbit so it keeps repeating.
A connection counts only if it holds in both directions.
All twenty-six selected connections did.
So the two branches join, at least in the region we sampled.
The plot made one family look like two.
Count it once.
Separately, two stability changes were re-checked at sixty-digit precision.

This is a preprint, not peer reviewed.
So what do all five examples share?

### 17 Five fields, one loop, three rules.

0:45, starts 13:35

Back to our map.
Five fields, one loop.
Each started from a shared state, ran alternatives, and needed a check.
**Compute once**: the fuzzer sets up its target once.
Each research attempt starts from the best commit.
**Count once**: duplicate crashes and six guides for one gene.
The two apparent branches were one orbit family.
**Publish once**: a fixed evaluator decides keep or revert.
A selector must still pick one patch.
**Compute once**.
**Count once**.
**Publish once**.
**Rule one**: compute once.

## 3 Runtime: compute once

14:20–17:45

### 18 The next step decides which state to reuse.

1:05, starts 14:20

What must the reused state hold?
The next step decides.
If it edits code and rebuilds, the cache from earlier supplies the files.
If it continues a live run, we restore a process or VM checkpoint.
We get only the memory that mechanism keeps.
If it needs an earlier answer, replay returns the recording.
Today we run restore fan-out on islo, our sandbox service.
That means many restores of one named snapshot.
Two hundred fifty-five of two hundred fifty-six restore-run-capture calls succeeded.
Whether live memory survives is untested until Gate zero, our first preregistered probe.
So pick the cheapest mechanism that supplies what the next step reads.
For a step that only reads files, a restore and a warm cache compete.

### 19 A restore pays only if it beats a warm cache.

1:15, starts 15:25

Here is the rule.
With N alternative runs, each would repeat a setup cost, P.
Preparing once avoids N minus one setups.
Against that, we pay H to save the state once.
Each child pays R to restart and D to keep its writes private.
Sandbox starts are already cheap.
A public benchmark puts them as low as seventy-three milliseconds.
So P must be real preparation.

**[ASK]** A question for you: which wins, a snapshot restore or a warm-cache rebuild? [PAUSE]
**[CLICK]** Take eight runs, H of two, R of one, D of three.
That is thirty-four resource-seconds of overhead.
With a cold setup of thirty, restoring avoids two hundred ten.
So it saves a hundred seventy-six.
**[CLICK]** With a warm cache, setup drops to two.
Restoring avoids only fourteen, so it loses twenty.
The warm cache wins: never build the same thing twice. [PAUSE]
But even a winning restore cannot undo everything.

### 20 A restore cannot undo remote effects, and a fork must not hold the right to publish.

1:05, starts 16:40

First, remote effects.
Save a sandbox, call a hosted model, then restore.
The sandbox is back, but the model has already answered and charged us.
A restore cannot rewind another service.
So an effect is not computed once, but recorded once, outside the child.
Our proposed design retries with an idempotency key.
The controller gives each child its own key, so forks never share one.
Second, credentials.
A fork copies every credential inside the sandbox.
So the publish key must stay outside.
The worker returns a patch and its test records.
Only the controller holds that key.
Our factory runs exercised that separation.
**Rule two**: count once.

## 4 Evidence: count once

**17:45–22:25**

### 21 A receipt is evidence's cache entry, written outside the child.

1:05, starts 17:45

Counting once starts with a receipt.
A receipt records what each trial did.
It works like a cache entry.
The key is what ran, on which bytes, from which inputs.
A hash names the bytes, as in the cache.
The value is the result and who reported it.
In our design, the harness writes it, outside the child.
A child's own report is only a claim.
Our factory logs show why.
A CI job passed in three seconds, but its evaluation was skipped.
A helper's code hash was recorded, but its source was not in the repository.
A hash is a fingerprint, not a backup.
An approval was labelled human, but the harness had answered it.
The inputs field lets a shared observation count once.

Next, we combine receipts.

## 22 Each fan-in operation needs its own check.

1:00, starts 18:50

Combining receipts is called fan-in, and it is not one operation.
If the trials tried different ideas, we select one.
Recall the patch selector in the software example.
Selection is itself a search.
So we recheck the winner on fresh data.
If the trials edited different parts of a program, we compose their edits.
The merged version needs its own test.
A made-up example: each change starts a one-gigabyte worker on a two-gigabyte server.
Each change fits.
Every pair fits.
All three together fail, and no pair test catches that.
If the trials estimated one quantity, we pool them.
The weights must account for errors they share.
So first, what do trials share?

## 23 Overlapping trials add less than their count.

0:50, starts 19:50

The simplest thing trials share is an observation.
Trial A returns a, b and c.
Trial B returns b, c and d.
That is four distinct observations, not six, so B adds only d.
We have seen this before.
The Roadbook counts a road once, however many drives cross it.
Six guides aimed at one gene are six looks at one gene.
Our reducer pools the results.
Today, it rejects a duplicate observation ID.
It also keeps each result's source history.
But different observations can still share errors.

## 24 At error correlation 0.1, a mean of 100 estimates is as precise as about nine.

0:50, starts 20:40

Take a hundred equally noisy estimates.
Their errors have pairwise correlation point one.
They might share a parent state, a seed or a test fixture.
Their average is as precise as about nine independent ones, not a hundred.
Shared errors count for less.
A shared bias does not average away at all.
Our reducer still assumes independence.
We propose weights that use the error covariance.
We also propose refusing a precision claim when the dependence is unknown.
A worker that overstates its own precision is worse than a shared error.

## 25 A worker that overstates its precision can take over the average.

0:55, starts 21:30

A weighted average multiplies each estimate by its precision, one over its variance.
So a worker claiming little uncertainty gets a large weight.
In our synthetic test, the true value is five.
One worker sends two thousand tight points near seventeen.
It claims fifty times its real precision.
Counting points would not catch it.
If we trust that precision, the average is seventeen point zero zero zero four.
One worker took over.
**[CLICK]** Our heuristic caps that precision and down-weights far-off estimates.
That gives four point nine five six six.
One synthetic case, not a Byzantine guarantee.
So check outside the child, including its error bars.
**Rule three**: publish once.

# 5 Acceptance: publish once

**22:25–25:40**

## 26 Our factory's patch passed every check it had.

0:55, starts 22:25

Start with our own factory run.
It was serial, on Docker sandboxes.
Nothing forked.
A repair.
Review blocked it and asked for tests.
Then a second repair, and approval.
After each repair, the harness ran the full suite.
1,972 cases, including skips, zero failures.
The repairs added a code path for retries.
If another attempt of the same job holds the workspace, the retry adopts it.
The promise: the job starts once.
No test forces that path.
**[CLICK]** Review approved, suite green, promise untested. [PAUSE]
I reverted that path by hand.
The PR merged on 3 October without it.
A person caught what the gate should catch. [PAUSE]
What should that gate be?

## 27 Accept only bytes whose promise was tested.

0:50, starts 23:20

Three steps.
First, freeze the candidate and its hash.
The hash that keyed the cache now names the bytes the controller may publish.
It proves identity, not correctness.
Second, require a test for each promise the change makes.
Run it on those exact files.
Here, force the retry onto a busy workspace and assert one dispatch.
Third, the controller publishes only those bytes.

That hash-bound publish check is built, but the promise test is proposed.
No test for the promise, no acceptance: this gate would have refused our patch. [PAUSE]
A checked result also has to survive.

### 28 Our second factory run lost an approved patch.

0:45, starts 24:10

In a second factory run, review approved a patch.
But delivery was refused three times, over a stray file.
That file was the harness's own review archive.
Then cleanup deleted the workspace, and the approved patch with it.
The fix is an order.
Save a copy outside the child, confirm the copy, then delete.
A hash alone is not a copy.
The order is proposed.
The loss was real.
But saving is not permission to publish.
A replaced or restored worker may still try to deliver.

### 29 The controller rejects a late result from a replaced worker.

0:45, starts 24:55

So the controller keeps an epoch, a permission version number.
Replacing a worker moves the epoch from one to two.
The old worker's late result still carries one, so the controller rejects it.
The check lives in the controller.
That way, restoring a snapshot cannot revive an old permission.
We also key each publish by its hash.
Then a retry is a no-op, like a cache hit.
Both are proposed.
One controller, current epoch, publish once.
What is still open?

## 6 Open questions

**25:40–26:45**

### 30 Five questions remain open; the preregistered tests have not started.

1:05, starts 25:40

Five questions remain open.
Three have preregistered tests.
None has started.
**Compute once**.
First comes Gate zero: does a restored snapshot keep live memory?
VM checkpoints can, but we have not shown ours does.
A value held only in RAM must survive into three restored children.
Until it passes, I say restore fan-out, not memory fork.
Second, does restoring beat a warm cache at three, six and twelve sandboxes?
**Count once**.
Third, do sibling restores fail a race test together more than unrelated ones?

That measures the shared environment, not agents' judgments.
Fourth, a design question: when should the reducer refuse a precision claim?
**Publish once**.
Fifth, does the epoch fence hold across restores?
Our TLA+ model is not yet model-checked.
Now, what this talk contributes.

# 7 Conclusion

**26:45–27:45**

## 31 Contributions: three rules for doing work once.

1:00, starts 26:45

My contributions, on the same map we started with.
**Compute once**: a cost rule.
Restore saved state only when the setup it avoids costs more than its overhead.
Price it against a warm cache.
**Count once**: an evidence contract.
The reducer I built rejects duplicate observation IDs.
Receipts kept outside the child and weights for shared errors are proposed.
**Publish once**: an acceptance gate.
The controller holds the publish key.
Its hash-bound publish check is built.
A test for every promise and the epoch fence are proposed.
One hash threads all three.
It keys the cache, binds the receipt, and names the published bytes.
It proves identity, not correctness.
**Compute once**.
**Count once**.
**Publish once**.
Your questions, please.

*End: 27:45. Leave the contributions slide up for questions.*